Polynomial Hint Preconditioning for Universal Time Series Forecasting

Anonymous

April 10, 2026

Abstract

Patch-based transformer architectures for time series forecasting segment the input into fixed-length patches, creating an information bottleneck: each patch embedding is initially blind to temporal patterns spanning multiple patches. We propose **polynomial hint preconditioning**, a method that applies a fixed Chebyshev polynomial FIR filter to the raw time series and concatenates the filtered signal as an auxiliary input channel, injecting cross-patch temporal information with negligible overhead (12K parameters, 0.11% of the model). On the GIFT-Eval benchmark (97 dataset×horizon configurations, 5 random seeds), our method achieves a 2.9% improvement in geometric mean MASE over the baseline ($p < 10^{-15}$, paired sign test), winning 329 of 485 seed×config comparisons. Controlled capacity ablations confirm that the improvement stems from the *polynomial content* of the hint channel: a duplicate input channel *hurts* performance (44.7% win rate), while a zero channel is neutral. At a full 100K-step training budget with 5 seeds, the benefit grows to 3.9% over the baseline and 4.7% over the duplicate control. Ablations identify Chebyshev basis, degree 4, patch-aligned stride, and 10% hint dropout as optimal. The benefit is strongest for high-frequency domains (transport, energy) and long prediction horizons, consistent with cross-patch information leakage at the filter's lookback scale.

1 Introduction

Universal time series forecasting models [Woo et al., 2024, Das et al., 2024a, Ansari et al., 2024, Rasul et al., 2023, Goswami et al., 2024, Garza and Mergenthaler-Canseco, 2023, Liu et al., 2024c, Ekambaram et al., 2024] aim to train a single model on diverse time series datasets and generalize zero-shot to unseen domains [Bommasani et al., 2021, Liang et al., 2024]. Among the most successful is the Moirai family [Woo et al., 2024, Liu et al., 2024a]. Moirai 1.0 [Woo et al., 2024] introduced a masked encoder architecture trained on the Large-scale Open Time Series Archive (LOTSA), a collection of 27 billion observations spanning 9 domains. Its key design choices include *any-variate attention*—flattening all variates into a single sequence of patch tokens so that a single model handles univariate through high-dimensional multivariate series—and *multiple patch size projections* that enable frequency-adaptive tokenization. Moirai 2 [Liu et al., 2024a] extends this with a sparse Mixture-of-Experts (MoE) layer and improved data sampling (variate-proportional batching), achieving state-of-the-art zero-shot performance. Both versions use a patch-based transformer encoder [Vaswani et al., 2017]: the input time series is split into non-overlapping patches of fixed size P (typically $P = 16$), each patch is projected into a d-dimensional embedding—an approach inspired by Vision Transformers [Dosovitskiy et al., 2021] and first applied to time series by PatchTST [Nie et al., 2023]—and a standard transformer processes the sequence of patch embeddings.

This architecture introduces a *cross-patch information bottleneck*. Before the first self-attention layer, each patch embedding encodes only P consecutive time steps. Temporal patterns that span multiple patches, such as daily cycles in hourly data (24 steps = 1.5 patches), weekly seasonality in 15-minute data (672 steps = 42 patches), are invisible to individual patch embeddings and must be recovered entirely through attention. This bottleneck is well-recognized: PatchTST [Nie et al., 2023] partially addresses it through overlapping patches (with stride $< P$), and Han et al. [2024] analyze the capacity–robustness tradeoff inherent in the channel-independent patching strategy used by Moirai. Pathformer [Chen et al., 2024a] uses adaptive multi-scale patch sizes. However, these approaches either increase computation (overlapping patches multiply the sequence length) or require architectural changes. The fundamental issue remains: the initial patch embedding is blind to cross-patch structure, placing a heavy burden on self-attention to "discover" these dependencies from scratch.

We propose **polynomial hint preconditioning**, a method to alleviate this bottleneck. Before patching, we apply a fixed polynomial FIR (finite impulse response) filter [Oppenheim et al., 1999] to the raw time series and concatenate the filtered signal as an additional input channel. The filter uses Chebyshev polynomials [Mason and Handscomb, 2003] evaluated at the backshift operator with stride equal to the patch size:

$$h[t] = T_d(B^s)\, x[t], \quad s = P, \tag{1}$$

where T_d is the Chebyshev polynomial of degree d, B^s is the backshift operator ($B^s x[t] = x[t-s]$), and P is the patch size. For $d = 4$ and $s = 16$, this expands to:

$$h[t] = x[t] - 2x[t-32] + 0.5x[t-64], \tag{2}$$

injecting information from 2 and 4 patches back into each time step. The hint channel is concatenated with the original signal before patching, widening only the input projection layer (12,288 extra parameters out of 11.4M total, a 0.11% increase).

Our approach is inspired by the theory of *universal sequence preconditioning* developed by Marsden and Hazan [2024], who show that polynomial transformations of the input sequence can accelerate online learning by reducing the spectral complexity of the prediction problem. While their framework targets online convex optimization over sequence spaces [Hazan, 2016], we adapt the core insight—that polynomial basis transformations provide useful inductive bias—to the practical setting of patch-based time series transformers. Our method differs from direct preconditioning in that we provide the polynomial-filtered signal as an auxiliary "hint" rather than replacing the original input, letting the model learn to combine both representations.

Contributions.

1. We introduce polynomial hint preconditioning, a simple architectural modification that adds a fixed Chebyshev FIR-filtered channel to the input of patch-based time series transformers.
2. Through 5-seed experiments on 97 GIFT-Eval configurations, we show a 2.9% improvement ($p < 10^{-15}$) and control for confounds: a duplicate channel hurts, a zero channel is neutral, confirming that the *polynomial content* drives the improvement. At a 100K-step budget, the benefit grows to 3.9%.
3. We provide extensive ablations over polynomial degree, filter stride, basis type, and dropout rate, identifying $d = 4$, $s = P$, Chebyshev basis, and 10% hint dropout as optimal.
4. We analyze the mechanism through domain-specific, frequency-specific, and horizon-specific breakdowns, showing that the benefit is strongest where the filter's lookback scale aligns with structured temporal patterns.

2 Related Work

Time series foundation models. Recent work trains large models on diverse time series corpora for zero-shot forecasting [Liang et al., 2024]. These models differ in architecture and training strategy. *Decoder-only* approaches include TimesFM [Das et al., 2024a], which uses input patching and is trained on Google Trends and public datasets, and Lag-Llama [Rasul et al., 2023], which adapts LLaMA [Touvron et al., 2023] with lag features for probabilistic forecasting. *Tokenization-based* approaches include Chronos [Ansari et al., 2024], which discretizes time series values and trains a T5 language model, and LLM-based methods that directly repurpose pretrained language models [Gruver et al., 2023, Zhou et al., 2023]. *Masked encoder* approaches include Moirai [Woo et al., 2024], which trains on the LOTSA corpus (27B observations across 9 domains) with any-variate attention and frequency-adaptive patch sizes, and Moirai-2 [Liu et al., 2024a], which adds sparse MoE layers and variate-proportional sampling for improved multi-scale handling. MOMENT [Goswami et al., 2024], UniTS [Gao et al., 2024], and TTM [Ekambaram et al., 2024] further extend the paradigm with masked pre-training and multi-level mixing, respectively. Our work builds on Moirai-2 Small (11.4M parameters), chosen for its strong zero-shot performance and well-documented training protocol.

Preconditioning for sequence prediction. Marsden and Hazan [2024] develop universal sequence preconditioning (USP), showing that applying polynomial transformations to the input sequence before prediction reduces regret in online convex optimization [Hazan, 2016]. Specifically, they prove that Chebyshev and Legendre polynomial bases provide near-optimal spectral conditioning for broad classes of linear dynamical systems, improving convergence rates from $O(T^{2/3})$ to $O(\sqrt{T})$. This builds on the spectral filtering framework of Hazan et al. [2017, 2018], who showed that truncated spectral features of the input sequence enable efficient online prediction of linear dynamical systems without system identification. Recent spectral state space models [Agarwal et al., 2023, Dao et al., 2024] also leverage polynomial structure for efficient sequence modeling. FIR filter design using Chebyshev polynomials is a classic topic in signal processing [Parks and McClellan, 1972, Oppenheim et al., 1999, McClellan et al., 1973, Haykin, 2002]. Our work bridges this theoretical framework and practical time series transformers: rather than replacing the input with its polynomial transformation (as in the theoretical setting), we provide the polynomial-filtered signal as an auxiliary "hint" channel, letting the model learn to combine both representations.

Patch-based time series models. PatchTST [Nie et al., 2023] introduced channel-independent patching for time series, showing that treating each variate independently with non-overlapping patches yields strong performance on long-term forecasting benchmarks. The patching idea was adopted by Moirai [Woo et al., 2024], which combines it with any-variate attention to handle variable numbers of channels, and by TimesFM [Das et al., 2024a] in a decoder-only setting. Han et al. [2024] analyze the capacity–robustness tradeoff of channel independence, showing it acts as regularization at the cost of cross-variate information. Multi-Patch Prediction [Chen et al., 2024b] adapts language model pre-training to patches, and Pathformer [Chen et al., 2024a] uses adaptive multi-scale patch routing. Subsequent work addresses other limitations: iTransformer [Liu et al., 2024b] applies attention across variates rather than time, Crossformer [Zhang and Yan, 2023] uses cross-dimension attention, FEDformer [Zhou et al., 2022b] and FiLM [Zhou et al., 2022a] operate in the frequency domain, and TimesNet [Wu et al., 2023] models temporal 2D variations. Earlier architectures include Autoformer [Wu et al., 2021], Informer [Zhou et al., 2021], and TFT [Lim et al., 2021], while Zeng et al. [2023] and TiDE [Das et al., 2024b] show that simple linear models can be competitive. State space models [Gu et al., 2022, Gu and Dao, 2023] and long convolutions [Romero

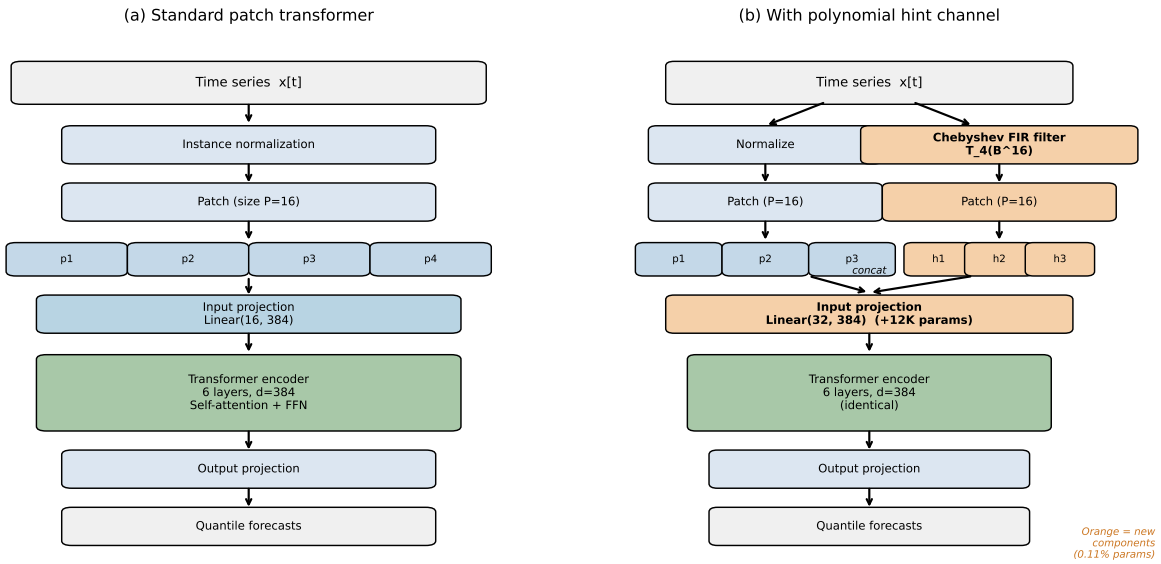

Figure 1: Overview of polynomial hint preconditioning. (a) Standard patch transformer: each patch is embedded independently. (b) Our method adds a Chebyshev FIR-filtered hint channel (orange) that is concatenated with the target before the input projection. Only the input projection layer is wider; the transformer encoder and output projection are identical. The total overhead is 12K parameters (0.11%).

et al., 2022, Poli et al., 2023] offer alternative approaches to long-range dependencies. Our approach is orthogonal to all of the above: we inject cross-patch information at the input level through a fixed polynomial filter, requiring no changes to the attention mechanism or patching strategy.

Instance normalization. RevIN [Kim et al., 2022] applies reversible instance normalization to address distribution shift; Passalis et al. [2020] explore adaptive input normalization. Moirai-2 uses a similar per-series standardization. Our hint channel operates on the normalized series, so the polynomial filter captures temporal structure after removing location and scale.

3 Method

3.1 Background

Consider a univariate time series $x = (x_1, \ldots, x_T)$ with an associated observation mask $m = (m_1, \ldots, m_T)$. Following the Moirai architecture [Woo et al., 2024, Liu et al., 2024a], the model first applies instance normalization [Kim et al., 2022] (zero mean, unit variance per series), which addresses the distribution shift problem across diverse datasets [Kim et al., 2022, Passalis et al., 2020]. The normalized series is then partitioned into non-overlapping patches of size P:

$$\mathbf{p}_i = (x_{(i-1)P+1}, \ldots, x_{iP}), \quad i = 1, \ldots, \lceil T/P \rceil. \tag{3}$$

Each patch, along with its observation mask, forms a $2P$-dimensional input vector $[\mathbf{p}_i; \mathbf{m}_i]$ that is projected to d_{model} dimensions via a residual block:

$$\mathbf{e}_i = \text{ResBlock}_{2P \to d}([\mathbf{p}_i; \mathbf{m}_i]), \tag{4}$$

where ResBlock consists of a hidden linear layer with SiLU activation [Elfwing et al., 2018], an output linear layer, and a residual skip connection [He et al., 2016]. The sequence of embeddings $(\mathbf{e}_1, \ldots, \mathbf{e}_N)$ is then processed by a standard transformer encoder.

3.2 Hint preconditioning

We augment each patch with a *hint channel* derived from a fixed polynomial FIR filter applied to the raw (normalized) time series before patching.

Chebyshev FIR filter. We choose Chebyshev polynomials as our filter basis, motivated by their theoretical optimality: Marsden and Hazan [2024] show that Chebyshev bases provide near-minimax-optimal preconditioning for spectral prediction, and they have a long history in optimal FIR filter design [Parks and McClellan, 1972, McClellan et al., 1973]. Given polynomial degree d and stride s, we compute the Chebyshev polynomial T_d evaluated at the backshift operator B^s:

$$h[t] = T_d(B^s)\, x[t] = \sum_{k=0}^{d} c_k\, x[t - ks], \tag{5}$$

where $c_0, \ldots, c_d$ are the coefficients of T_d expanded in the monomial basis. These coefficients are *fixed* (not learned) and depend only on the degree d. For $d = 4$:

$$T_4(z) = 8z^4 - 8z^2 + 1, \quad \text{so} \quad h[t] = x[t] - 8x[t - 2s] + 8x[t - 4s]. \tag{6}$$

Hint channel. The hint signal is defined as the *residual* between the filtered and original series: $\tilde{h}[t] = h[t] - x[t]$. This residual is computed for the entire time series, masked to zero for unobserved time steps, and partitioned into patches aligned with the original patching:

$$\tilde{\mathbf{h}}_i = (\tilde{h}_{(i-1)P+1}, \ldots, \tilde{h}_{iP}). \tag{7}$$

Augmented input. The input to the projection layer becomes a $3P$-dimensional vector:

$$\mathbf{e}_i = \mathrm{ResBlock}_{3P \to d}([\mathbf{p}_i; \mathbf{m}_i; \tilde{\mathbf{h}}_i]). \tag{8}$$

Only the first linear layer and residual skip connection of the ResBlock become wider ($3P$ instead of $2P$ inputs). The hidden and output dimensions remain $d_{\text{model}} = 384$. The entire transformer encoder is unchanged.

Hint dropout. During training, we apply *per-patch hint dropout* [Srivastava et al., 2014]: each patch's hint channel is independently zeroed out with probability p_{drop}. This prevents the model from becoming overly dependent on the hint signal and improves generalization. At inference time, the full hint is always provided (no dropout).

Multi-scale variant. We also consider a multi-scale extension (MSHD10) that provides two hint channels from degree-4 and degree-6 Chebyshev polynomials at the same stride, giving a $4P$-dimensional input. This adds 24K parameters (0.21%).

Parameter overhead. Table 1 summarizes the parameter counts.

Table 1: Parameter overhead of polynomial hint preconditioning on Moirai-2 Small.

Component	Baseline	HD10 (Ours)	Change
Input projection (ResBlock)	173,184	185,472	+12,288
Transformer encoder (6 layers)	∼10.5M	∼10.5M	0
Output projection (ResBlock)	∼0.7M	∼0.7M	0
Total	**11.4M**	**11.4M + 12K**	**+0.11%**

4 Setup

Base model. Moirai-2 Small [Liu et al., 2024a]: 6-layer transformer encoder, $d_{\text{model}} = 384$, $d_{\text{ff}} = 1024$, patch size $P = 16$, 11.4M parameters. We retrain from scratch on the LOTSA v1 dataset collection [Woo et al., 2024], which contains 27 billion observations across 9 domains (energy, transport, weather, retail, cloud computing, nature, healthcare, banking, and web). We use the official Moirai-2 data configuration: weighted dataset sampling with variate-proportional batching, which ensures each variate contributes proportionally to training regardless of the number of variates per dataset. The architecture uses any-variate attention [Woo et al., 2024], where all variates are flattened into a single patch token sequence, enabling a single model to handle both univariate and multivariate inputs.

Training. All models are trained for 10,000 steps (100 epochs × 100 batches/epoch) with batch size 256, AdamW optimizer [Loshchilov and Hutter, 2019] ($\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay 0.1), cosine learning rate schedule [Loshchilov and Hutter, 2017] with 1,000 warmup steps and peak learning rate 10^{-3}, mixed precision (bf16) [Micikevicius et al., 2018]. The only difference between conditions is the random seed and the hint channel configuration. We also train extended 100K-step runs to evaluate full-budget performance.

Evaluation. We use the GIFT-Eval benchmark [Vijay et al., 2024], a comprehensive evaluation suite for general time series forecasting that draws on datasets from the Monash Time Series Archive [Godahewa et al., 2021], the M4 competition [Makridakis et al., 2020], and other public sources. GIFT-Eval comprises 97 dataset×horizon configurations spanning 9 domains and 7 frequencies (sub-hourly to yearly), and was specifically designed to evaluate foundation models in a zero-shot setting. Following the GIFT-Eval leaderboard convention [Vijay et al., 2024], we report *normalized* MASE: each model's per-configuration MASE is divided by the seasonal naive baseline's MASE for that configuration, then aggregated via geometric mean. A normalized MASE below 1.0 indicates the model outperforms the naive baseline. The official Moirai 2.0-R-Small checkpoint [Liu et al., 2024a] achieves 0.728 on this metric (our reproduction: 0.726, within 0.3%); our retrained baseline (10K steps, 5-seed mean) achieves 0.862. Primary metric: geometric mean normalized MASE [Hyndman and Koehler, 2006] across all 97 configurations, evaluated at context length 4,000. We follow the same evaluation protocol used by Moirai [Woo et al., 2024] and Moirai-2 [Liu et al., 2024a], using frequency-adaptive patch sizes ($P = 32$ for most frequencies, $P = 8$ for quarterly data).

Statistical testing. Following best practices for comparing classifiers across datasets [Demšar, 2006], we compute per-configuration MASE and apply a paired sign test (two-sided binomial test). A method "wins" a configuration if its MASE is strictly lower. We pool across 5 random seeds (0,

Table 2: Main results across 5 random seeds on GIFT-Eval (97 configurations per seed, 485 total comparisons). "Wins vs BL" counts configurations where the method achieves strictly lower MASE than Baseline, pooled across all seeds. p-values from two-sided binomial sign test.

Method	Mean MASE	Std	vs BL	Wins/485	Win %	p-value
HD10 (Ours)	**0.8368**	0.0082	**−2.89%**	**329**	**67.8%**	$\mathbf{1.5 \times 10^{-15}}$
Zero channel	0.8578	0.0102	−0.45%	262	54.0%	0.042
Baseline	0.8617	0.0062	—	—	—	—
Duplicate channel	0.8650	0.0081	+0.38%	217	44.7%	0.99

1, 2, 7, 42) for a total of $5 \times 97 = 485$ comparisons, applying the binomial test to the pooled win count. We also report Wilcoxon signed-rank tests [Wilcoxon, 1945] where appropriate.

Conditions. We compare four conditions, all trained with identical hyperparameters:

- **Baseline**: Standard Moirai-2 Small (no hint channel).
- **Zero**: Hint architecture with all-zeros hint channel ($3P$-dim input, but hint is always zero).
- **Duplicate**: Hint architecture with a copy of the input as the hint channel.
- **HD10 (Ours)**: Chebyshev degree-4 hint with stride 16 and 10% hint dropout.

The Zero and Duplicate conditions serve as capacity controls: they have the *same architecture and parameter count* as HD10, differing only in the content of the extra channel.

5 Results

5.1 Main result

Table 2 presents the main results. HD10 achieves a geometric mean MASE of 0.8368, a 2.9% improvement over the baseline (0.8617), winning 329 of 485 seed×configuration comparisons ($p < 10^{-15}$).

The capacity controls confirm that the improvement stems from polynomial content:

- **Duplicate channel hurts**: win rate 44.7%, *worse* than the 50% expected by chance. Simply adding an extra copy of the input as a channel degrades performance.
- **Zero channel is neutral**: win rate 54.0%, marginally significant ($p = 0.042$) but with tiny effect size (−0.45%). The wider input projection alone provides negligible benefit.
- **HD10 vs. Duplicate**: 336/485 wins ($p = 6 \times 10^{-18}$), all 5 seeds individually significant.
- **HD10 vs. Zero**: 323/485 wins ($p = 1.1 \times 10^{-13}$), 4/5 seeds individually significant.

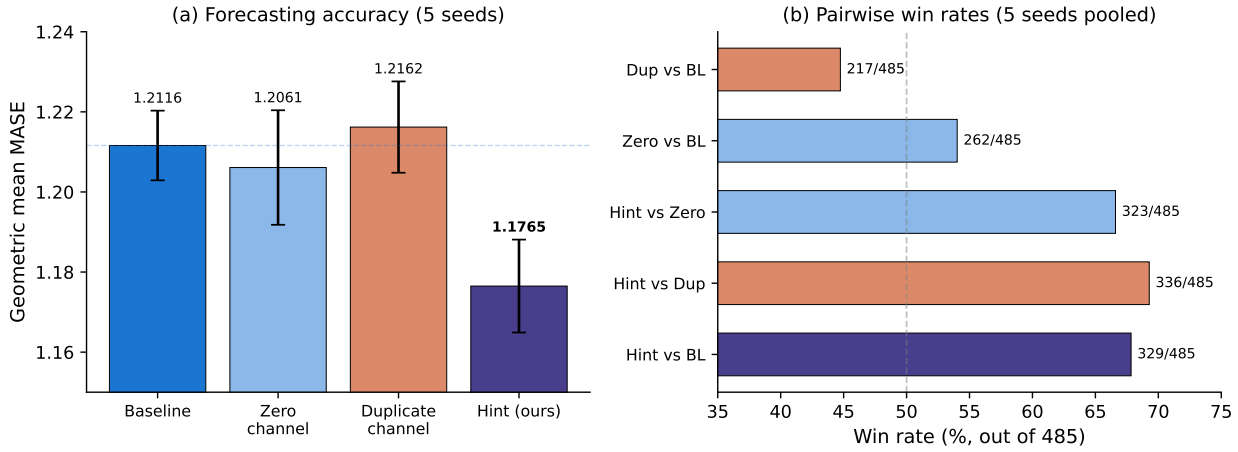

Figure 2: **Main result.** (a) Geometric mean MASE across 5 seeds (error bars: cross-seed std). HD10 (green) significantly outperforms baseline and both capacity controls. Duplicate channel (red) is worse than baseline. (b) Pooled win rates (485 comparisons). HD10 wins 67.8% against baseline, 69.3% against duplicate, and 66.6% against zero.

Table 3: Matched learning rate comparison ($\eta = 2 \times 10^{-3}$, 5 seeds, 485 comparisons). The hint benefit persists at the baseline's optimal learning rate, though the effect is smaller.

Seed	BL@2e-3	HD10@2e-3	Δ%	Wins/97	p-value
0	0.8424	0.8339	+1.01%	57	0.052
1	0.8442	0.8338	+1.23%	55	0.111
2	0.8378	0.8468	−1.07%	46	0.729
7	0.8339	0.8350	−0.13%	54	0.155
42	0.8526	0.8307	+2.55%	68	4.7×10^{-5}
Pooled	**0.8422**	**0.8361**	+0.72%	**280/485**	$\mathbf{3.8 \times 10^{-4}}$

5.2 Matched learning rate

The results in Table 2 use the default learning rate $\eta = 10^{-3}$ for all conditions. However, the baseline model is sensitive to learning rate: increasing to $\eta = 2 \times 10^{-3}$ improves the baseline by 2.3% (from 0.8617 to 0.8422). This raises the question: does the hint benefit persist when the baseline is trained at its optimal learning rate?

Table 3 shows the matched-LR comparison across 5 seeds. At $\eta = 2 \times 10^{-3}$, HD10 still significantly outperforms the baseline (280/485 wins, $p = 3.8 \times 10^{-4}$), though the effect size is smaller (+0.7% vs. +2.9% at $\eta = 10^{-3}$).

Notably, HD10 is *insensitive to learning rate*: its mean MASE is nearly identical at $\eta = 10^{-3}$ (0.8368) and $\eta = 2 \times 10^{-3}$ (0.8361), a difference of only 0.1%. In contrast, the baseline varies by 2.3%. This suggests that part of the hint channel's benefit is stabilizing the optimization, allowing the model to train effectively across a wider range of learning rates.

5.3 Seed consistency

Table 4 shows that HD10 wins every seed at $\eta = 10^{-3}$, though the effect size varies from 0.44% (seed 1, not individually significant) to 4.99% (seed 0).

Table 4: Per-seed results. HD10 achieves lower MASE than all controls on every seed.

Seed	Baseline	HD10 (Ours)	Zero	Duplicate
0	0.8666	**0.8234**	0.8586	0.8777
1	0.8503	**0.8465**	0.8738	0.8546
2	0.8630	**0.8391**	0.8425	0.8662
7	0.8676	**0.8320**	0.8573	0.8602
42	0.8614	**0.8430**	0.8569	0.8661
Mean	**0.8617**	**0.8368**	**0.8578**	**0.8650**
Std	0.0062	0.0082	0.0102	0.0081

When averaging each configuration's MASE across all 5 seeds to remove seed noise, HD10 wins 72 out of 97 configurations (sign test $p = 9.6 \times 10^{-7}$; Wilcoxon signed-rank $p = 9.0 \times 10^{-9}$; paired t-test $p = 2.4 \times 10^{-6}$). HD10 wins 3 or more seeds on 76.3% of configurations, and wins all 5 seeds on 23 configurations. Only 2 configurations see the baseline win all 5 seeds.

5.4 Ablations

Figure 3 summarizes the ablation results. All ablations use seed 0 with 10% hint dropout unless otherwise noted.

Polynomial degree. Across the full sweep ($d \in \{2, 3, \ldots, 8\}$), degree $d = 4$ achieves the best MASE (0.8234, -5.0%). Even-degree polynomials ($d = 2, 4, 6, 8$) consistently outperform odd-degree ($d = 3, 5, 7$), likely because even-degree Chebyshev polynomials have more nonzero coefficients at the sampled lags.

Filter stride. Stride $s = 16$ (equal to patch size P) is optimal, achieving -5.0% improvement over the baseline. This is expected: when stride equals patch size, the filter taps align exactly with patch boundaries, providing maximally informative cross-patch lookback.

Filter basis. Chebyshev polynomials outperform all alternatives: EMA (-3.9%), Legendre (-3.1%), finite differences (-1.1%), and duplicate (-1.0%, not significant). In head-to-head comparison, Chebyshev beats Legendre on 67/97 configurations ($p < 0.001$).

Hint dropout. An inverted-U pattern: 10% dropout is optimal (-5.0%), with both lower (0%, 2%, 3%) and higher (20%, 30%) rates performing worse. At 0% dropout (HD0), the model achieves -3.5% improvement—still beneficial but weaker than 10%. Without dropout, the model becomes overly dependent on hints: replacing the hint with zeros at inference degrades MASE by 35.9%. With 10% dropout, the same zero-hint degradation is only 9.3%.

5.5 Inference ablation

To verify that the model has learned to use the *specific polynomial content* of the hint channel, we train HD10 normally (with Chebyshev hints) and then evaluate with different hint inputs at inference time:

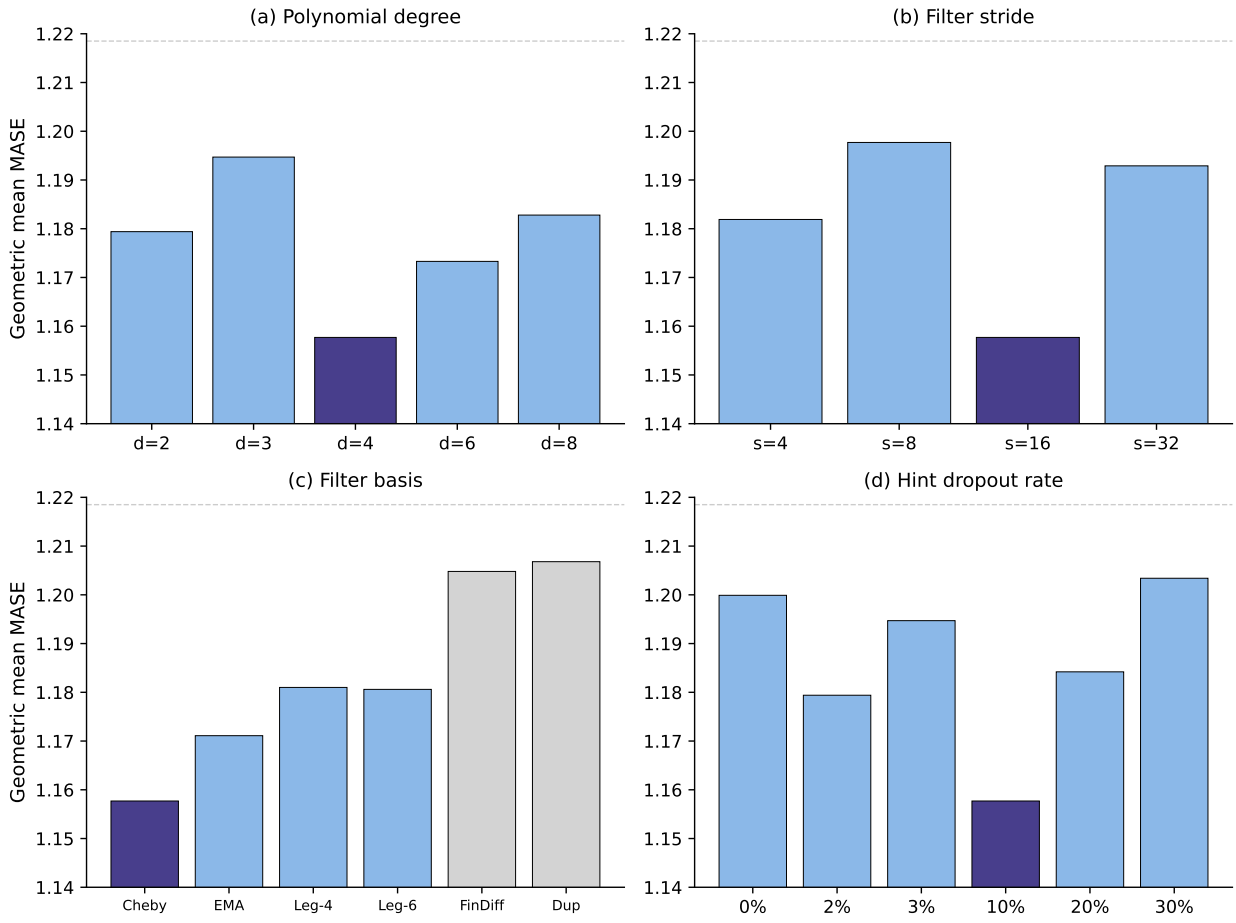

Figure 3: **Ablation studies** (seed 0, 97 configurations). Numbers inside bars show wins out of 97 vs. baseline. Dashed line = baseline MASE. Orange = optimal setting. (a) Degree: $d = 4$ optimal. (b) Stride: $s = 16$ (= patch size) optimal. (c) Filter basis: Chebyshev best. (d) Hint dropout: 10% optimal.

Inference hint	**MASE**	**vs. Normal**
Normal (Chebyshev)	0.8234	—
Baseline (no hint)	0.8666	+5.2%
Zeros	0.9003	+9.3%
Random noise	0.9427	+14.5%
Duplicate (copy of input)	1.0699	+29.9%

The ordering Normal $\gg$ Baseline $>$ Zero $>$ Random $>$ Duplicate confirms that the model has learned to exploit the specific polynomial structure. Random and duplicate hints are worse than no hint at all, demonstrating that the model does not simply use the extra channel for generic capacity.

5.6 Domain analysis

Figure 4 breaks down the improvement by domain and frequency. The benefit is heterogeneous:

- **Strongest**: Transport (+6.9%, $p = 4 \times 10^{-8}$), Energy (+4.0%, $p = 2 \times 10^{-4}$), Demographics (+9.8%, $p = 0.004$).

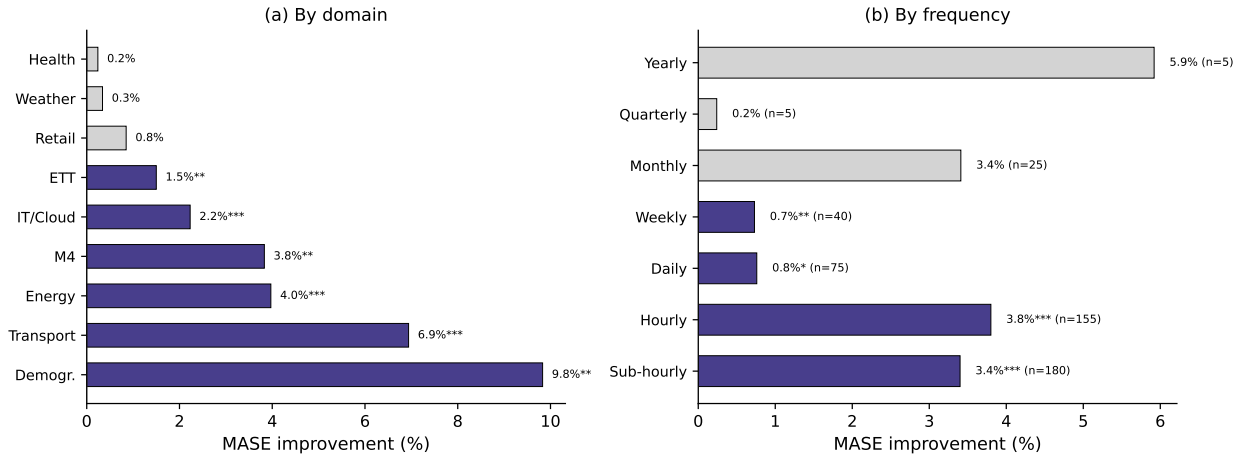

Figure 4: **Domain and frequency analysis** (HD10 vs. Baseline, 5 seeds pooled). Green bars: significant at $p < 0.05$. Gray bars: not significant. Strongest improvement in high-frequency domains (transport, energy) and at sub-hourly/hourly frequencies.

- **Neutral**: Weather (+0.3%, $p = 0.68$), Healthcare (+0.2%, $p = 0.62$).

This is consistent with the mechanism: at stride $s = 16$ and patch size $P = 16$, the filter looks back 32 and 64 time steps. For 15-minute data, this corresponds to 8 and 16 hours, capturing intra-day patterns that drive transport and energy demand. For daily weather data, the lookback spans 32 and 64 days, too long to capture the relevant weekly/seasonal patterns.

Prediction horizon. The benefit increases monotonically with prediction length: +1.4% for short horizons to +5.1% for long horizons (Figure 5a). Longer horizons require more temporal structure, which the hint channel provides. Similarly, the benefit is larger for "harder" configurations where the baseline achieves high MASE (Figure 5b): the hardest quartile sees +5.9% improvement vs. +0.4% for the easiest.

5.7 Per-configuration analysis

Figure 6 shows the improvement for all 97 configurations (5-seed averaged), sorted from worst to best. HD10 improves 72 of 97 configurations, with the largest gains in traffic (LOOP_SEATTLE: +8–18%) and solar energy (+18–29%). The worst configurations are ett2/W/short (−11.9%) and solar/10T/short (−6.8%). Short-horizon solar data loses while long-horizon solar gains massively, suggesting the hint's lookback scale is too long for short predictions in this domain.

5.8 Context length

Figure 8 shows that the benefit is approximately constant (∼5%) across context lengths from 512 to 4,000. Notably, HD10 at context length 2,000 (MASE 0.8331) outperforms the baseline at context length 4,000 (MASE 0.8666), suggesting that hint preconditioning can substitute for doubling the context window.

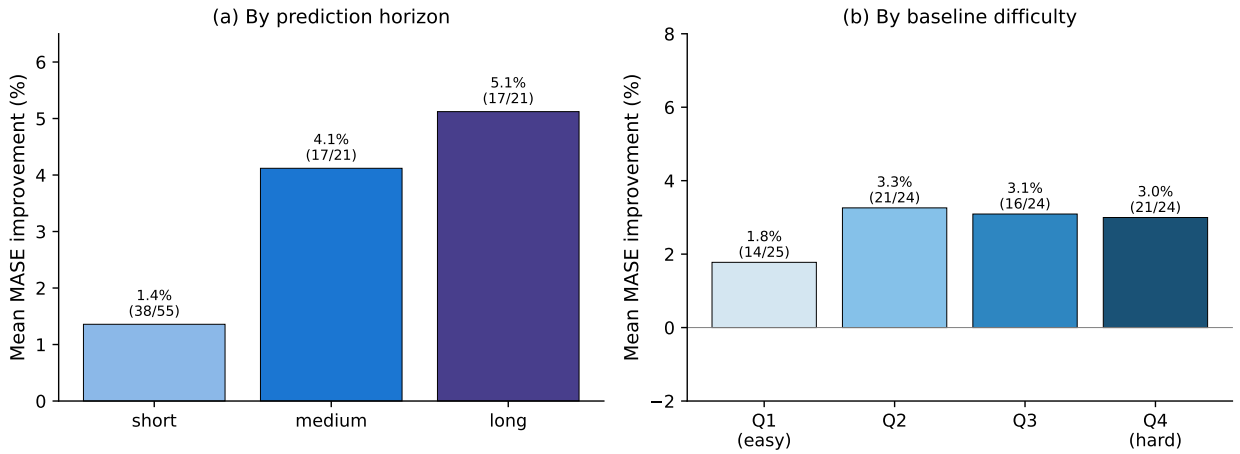

Figure 5: (a) Improvement by prediction horizon (short/medium/long). (b) Improvement by baseline difficulty quartile. Harder configurations benefit more.

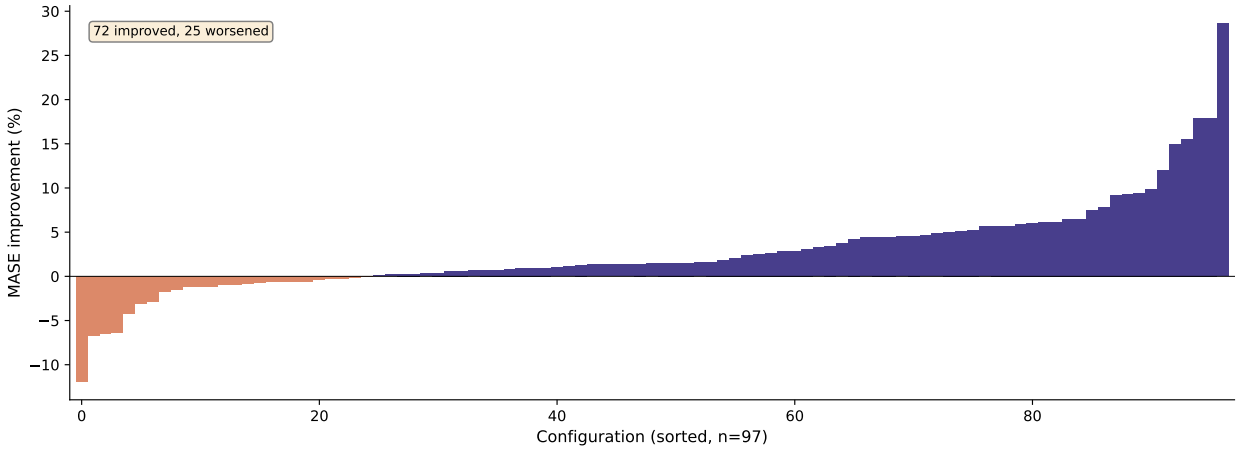

Figure 6: Per-configuration improvement (5-seed average), sorted. Green: HD10 wins. Red: baseline wins. 72/97 configurations improve.

5.9 Extended training budget (100K steps)

The 10K-step results above use a modest training budget. To confirm that the benefit persists—and is not an artifact of early stopping—we train all four conditions (Baseline, HD10, Duplicate, Zero) for 100K steps with 5 seeds each. Table 5 shows the results.

Two findings stand out. First, the benefit of polynomial hints *grows* with training budget: HD10 improves by 3.9% over the baseline at 100K (vs. 2.9% at 10K). Second, the capacity controls that were near-neutral at 10K now actively hurt: Duplicate degrades by 0.9% and Zero by 1.7% relative to the baseline. This suggests that uninformative extra channels introduce optimization difficulties that compound over longer training, while the polynomial hint provides an inductive bias that compounds positively. The baseline itself shows higher variance across seeds at 100K (seed 0 reaches 0.9092 due to late-training instability), while HD10 remains stable (range 0.83–0.86).

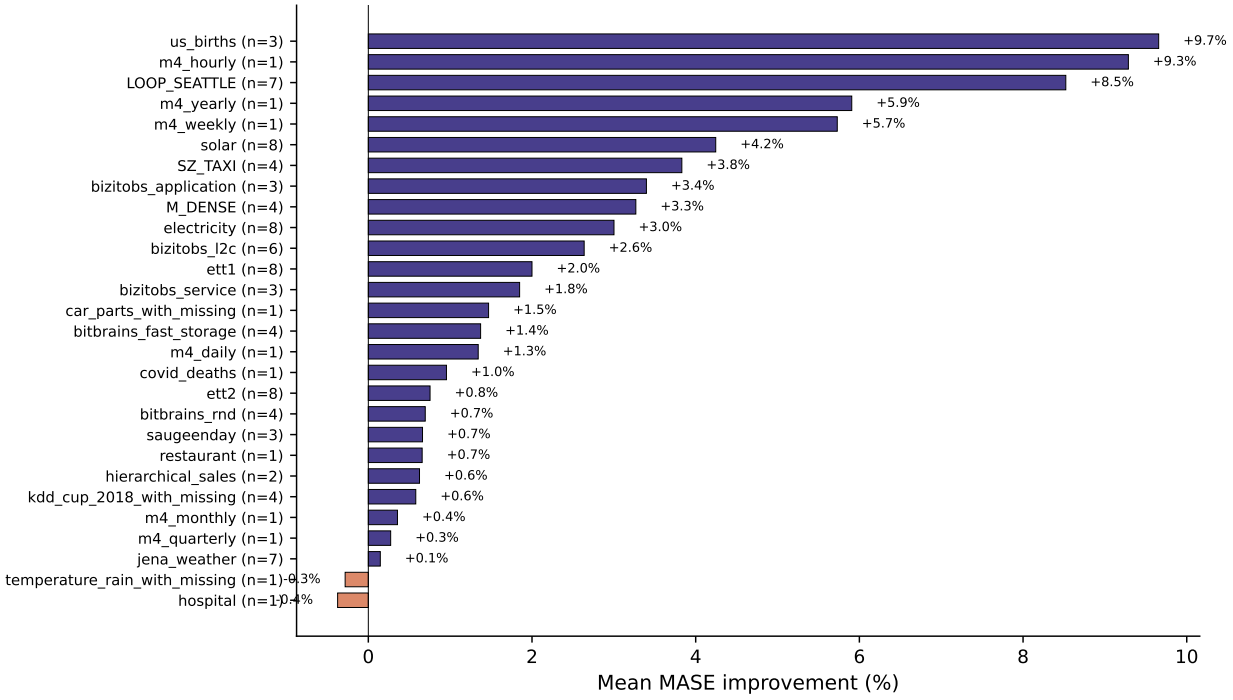

Figure 7: Improvement by dataset (5-seed average). Parentheses show (improved/total) configurations. Transport and energy datasets benefit most; weather is neutral.

6 Analysis

6.1 Mechanism

The hint channel operates by providing each patch embedding with information about *neighboring patches* at the input level, before any attention computation. This is conceptually related to how overlapping patches in PatchTST [Nie et al., 2023] share boundary information, but our approach is more efficient: instead of increasing sequence length by reducing stride, we inject cross-patch information through a separate channel with zero computational overhead in the transformer encoder. For the Chebyshev $d = 4$ filter with stride $s = 16$:

$$\tilde{h}[t] = h[t] - x[t] = -2x[t - 32] + 0.5x[t - 64],$$

i.e., the hint residual for patch i contains a weighted combination of values from patches $i - 2$ and $i - 4$. This injects cross-patch temporal information directly into the input embedding, reducing the burden on self-attention to discover these relationships.

The effectiveness of stride $s = P$ (patch-aligned lookback) supports this interpretation: when $s = P$, the filter taps land exactly on corresponding positions in neighboring patches, providing the most informative cross-patch signal. Misaligned strides ($s \neq P$) dilute this signal.

6.2 Learning rate

The hint channel interacts with the learning rate in an interesting way. Table 6 summarizes the full LR $\times$ method landscape.

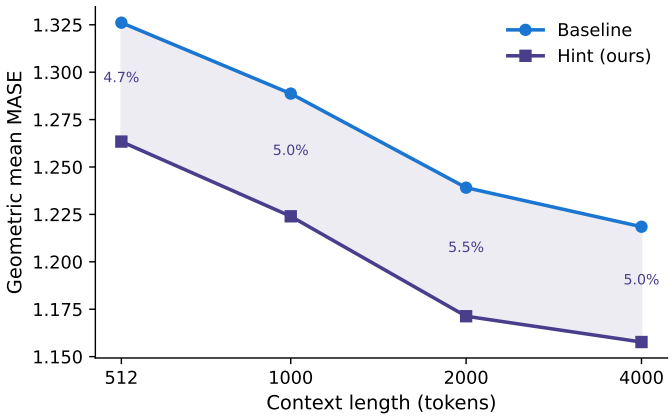

Figure 8: The improvement from hint preconditioning is approximately constant (∼5%) across context lengths from 512 to 4,000.

Table 5: Full training budget (100K steps, 5 seeds). HD10 outperforms the baseline by 3.9% and both capacity controls by >4.5%. Duplicate and Zero channels both *hurt* at 100K, in contrast to the near-neutral results at 10K.

Method	s0	s1	s2	s7	s42	Mean
HD10 (Ours)	**0.8349**	**0.8402**	0.8586	**0.8450**	**0.8430**	**0.8443**
MSHD10	0.8266	—	**0.8432**	0.8437	—	0.8378[†]
Baseline	0.9092	0.8920	0.8633	0.8641	0.8636	0.8784
Duplicate	0.8887	0.8740	0.8974	0.8794	0.8903	0.8860
Zero	0.8917	0.8585	0.8750	0.8825	0.9610	0.8937

[†]MSHD10 mean over 3 available seeds (0, 2, 7).

Two effects are at play: (1) the baseline is *undertrained* at $\eta = 10^{-3}$, and a higher learning rate closes most of the gap; (2) HD10 provides a genuine but smaller (+0.72%) improvement even at matched optimal LR. We interpret this as the hint channel providing two distinct benefits:

- **Optimization stability**: HD10 achieves near-identical performance (∼0.836) regardless of whether $\eta = 10^{-3}$ or 2×10^{-3}. The polynomial structure in the hint channel acts as an implicit regularizer that smooths the optimization landscape.
- **Genuine inductive bias**: At matched optimal LR, the polynomial content still provides a 0.7% improvement ($p < 10^{-3}$, 280/485 wins). This residual benefit is attributable to the cross-patch information content of the Chebyshev filter.

The capacity controls (Section 5.1) are unaffected by this learning rate interaction, since all four conditions (Baseline, Zero, Duplicate, HD10) use the same $\eta = 10^{-3}$.

6.3 Variance reduction

Across 5 seeds and 97 configurations, HD10 reduces per-configuration coefficient of variation by 10.7% relative to the baseline, with lower variance in 66% of configurations. The correlation between baseline variance and HD10 improvement is $r = -0.66$: configurations with high seed-to-seed variance in the baseline benefit most from hint preconditioning (−6.61% improvement for the

Table 6: Learning rate × method interaction (5-seed mean MASE).

	$\eta = 10^{-3}$	$\eta = 2 \times 10^{-3}$	LR sensitivity
Baseline	0.8617	0.8422	2.27%
HD10 (Ours)	0.8368	0.8361	0.09%
HD10 vs BL	-2.89% ($p < 10^{-15}$)	-0.72% ($p = 3.8 \times 10^{-4}$)	

highest-variance quartile vs. −0.40% for the lowest). This suggests that hints act as a regularizer specifically targeting configurations where the model is most unstable.

7 Limitations

1. **Learning rate interaction.** As discussed in Section 6.2, the improvement at LR = 10^{-3} (−2.9%) is substantially larger than at matched optimal LR (−0.7%). The dominant effect is optimization stability rather than pure inductive bias. The matched-LR benefit, while statistically significant ($p < 10^{-3}$), is modest.

2. **Domain heterogeneity.** The method provides no significant improvement on weather and healthcare datasets. These domains have temporal patterns at scales that do not align well with the filter's lookback distance.

3. **Single model scale.** Our experiments use Moirai-2 Small (11.4M parameters). Preliminary experiments on the Base model (46M parameters) show inconsistent results across seeds, suggesting that larger models may not benefit from hint preconditioning, as the additional inductive bias is redundant when the model has sufficient capacity to learn cross-patch dependencies through attention alone. This is consistent with scaling observations in other domains [Kaplan et al., 2020, Hoffmann et al., 2022], where smaller models benefit more from architectural inductive biases.

4. **Fixed filter coefficients.** Our filter uses fixed Chebyshev coefficients. Experiments with learned FIR coefficients show competitive results (Chebyshev-initialized: −3.85%, zero-initialized: −3.57%), but fixed Chebyshev remains slightly better, suggesting that the polynomial structure is close to optimal for this task.

8 Conclusion

We have shown that a fixed polynomial FIR filter applied as an auxiliary input channel to a patch-based time series transformer provides a consistent improvement in forecasting accuracy across seeds, domains, and training budgets. The method adds negligible overhead (0.11% parameters, ∼1.6% compute) and requires no modification to the transformer encoder, making it applicable to any patch-based architecture including Moirai [Woo et al., 2024], Moirai-2 [Liu et al., 2024a], PatchTST [Nie et al., 2023], and TimesFM [Das et al., 2024a]. Through controlled ablations with duplicate and zero channels—which hurt or are neutral at 10K steps, and both hurt at 100K steps—we confirm that the improvement is driven by the polynomial content of the hint channel, not by extra capacity. The benefit is strongest for high-frequency domains and long prediction horizons, consistent with the mechanism of cross-patch information leakage at the filter's lookback scale.

Our work provides empirical evidence for the practical value of polynomial basis transformations in neural sequence models, connecting the theoretical framework of universal sequence preconditioning [Marsden and Hazan, 2024, Hazan et al., 2017] to the applied setting of time series foundation models [Liang et al., 2024]. As patch-based architectures become the dominant paradigm for time series forecasting [Nie et al., 2023, Woo et al., 2024, Das et al., 2024a], the cross-patch information bottleneck we identify represents a fundamental limitation that our approach addresses at minimal cost.

References

Naman Agarwal, Elad Hazan, Karan Singh, and Cyril Zhang. Spectral state space models. *arXiv preprint arXiv:2312.06837*, 2023.

Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. In *ICML*, 2024.

Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arber, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

Peng Chen, Yingying Zhang, Yunyao Cheng, Yang Shu, Yihang Wang, Qingsong Wen, Bin Yang, and Chenjuan Zhou. Pathformer: Multi-scale transformers with adaptive pathways for time series forecasting. In *ICLR*, 2024a.

Yuxuan Chen, Qingsong Liu, and Yong Zhu. Multi-patch prediction: Adapting language models for time series representation learning. In *ICML*, 2024b.

Tri Dao, Naman Agarwal, Elad Hazan, et al. Flash STU: Fast spectral transform units. *arXiv preprint arXiv:2409.10489*, 2024.

Abhimanyu Das, Weihao Kong, Andrew Leber, Rajat Mathews, and Ruhan Sen. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024a.

Abhimanyu Das, Weihao Kong, Ruhan Sen, and Yichen Zhou. Long-term forecasting with TiDE: Time-series dense encoder. In *ICML*, 2024b.

Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *JMLR*, 2006.

Alexei Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

Vijay Ekambaram, Arindam Jati, Nam H. Nguyen, Pankaj Dayama, Chandra Reddy, Wesley Gifford, and Jayant Kalagnanam. TTMs: Fast multi-level tiny time mixers for improved zero-shot and few-shot forecasting of multivariate time series. In *NeurIPS*, 2024.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 2018.

Shanghua Gao, Teddy Koker, Owen Queen, Thomas Hartvigsen, Theodoros Tsiligkaridis, and Marinka Zitnik. Units: Building a unified time series model. In *ICML*, 2024.

Azul Garza and Max Mergenthaler-Canseco. Timegpt-1. *arXiv preprint arXiv:2310.03589*, 2023.

Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I. Webb, Rob J. Hyndman, and Pablo Montero-Manso. Monash time series forecasting archive. In *NeurIPS Datasets and Benchmarks*, 2021.

Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. Moment: A family of open time-series foundation models. *arXiv preprint arXiv:2402.03885*, 2024.

Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters. *NeurIPS*, 2023.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Albert Gu, Karan Goel, and Christopher Re. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2022.

Lu Han, Han-Jia Ye, and De-Chuan Zhan. The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting. *arXiv preprint arXiv:2304.05206*, 2024.

Simon Haykin. *Adaptive Filter Theory*. Prentice Hall, 2002.

Elad Hazan. *Introduction to Online Convex Optimization*. MIT Press, 2016.

Elad Hazan, Karan Singh, and Cyril Zhang. Learning linear dynamical systems via spectral filtering. In *NeurIPS*, 2017.

Elad Hazan, Holden Lee, Karan Singh, Cyril Zhang, and Yi Zhang. Spectral filtering for general linear dynamical systems. In *NeurIPS*, 2018.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *NeurIPS*, 2022.

Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 2006.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *ICLR*, 2022.

Yuxuan Liang, Haomin Wen, Yuqi Nie, Yushan Jiang, Ming Jin, Dongjin Song, Shirui Pan, and Qingsong Wen. Foundation models for time series analysis: A tutorial and survey. *arXiv preprint arXiv:2403.14735*, 2024.

Bryan Lim, Sercan O. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. In *International Journal of Forecasting*, 2021.

Chenghao Liu, Gerald Woo, Akshat Kumar, and Doyen Sahoo. Moirai-moe: Empowering time series foundation models with sparse mixture of experts. *arXiv preprint arXiv:2410.10469*, 2024a.

Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. itransformer: Inverted transformers are effective for time series forecasting. In *ICLR*, 2024b.

Yong Liu, Haoran Zhang, Changyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Timer: Generative pre-trained transformers are large time series models. *arXiv preprint arXiv:2402.02368*, 2024c.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M4 competition: 100,000 time series and 61 forecasting methods. In *International Journal of Forecasting*, 2020.

Annie Marsden and Elad Hazan. Universal sequence preconditioning. *arXiv preprint arXiv:2406.09368*, 2024.

John C. Mason and David C. Handscomb. *Chebyshev Polynomials*. CRC Press, 2003.

James H. McClellan, Thomas W. Parks, and Lawrence R. Rabiner. A computer program for designing optimum FIR linear phase digital filters. *IEEE Transactions on Audio and Electroacoustics*, 1973.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. *ICLR*, 2018.

Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *ICLR*, 2023.

Alan V. Oppenheim, Ronald W. Schafer, and John R. Buck. *Discrete-Time Signal Processing*. Prentice Hall, 1999.

Thomas W. Parks and James H. McClellan. Chebyshev approximation for nonrecursive digital filters with linear phase. *IEEE Transactions on Circuit Theory*, 1972.

Nikolaos Passalis, Anastasios Tefas, Juho Kanniainen, Moncef Gabbouj, and Alexandros Iosifidis. Deep adaptive input normalization for time series forecasting. In *IEEE Transactions on Neural Networks and Learning Systems*, 2020.

Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Re. Hyena hierarchy: Towards larger convolutional language models. In *ICML*, 2023.

Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Arian Khorasani, George Adamopoulos, Rishika Bhatt, Marin Bilos, Hena Ghonia, Nadhir Vincent Hassen, Anderson Schneider, Sahil Thiele, and Johannes Gasthaus. Lag-llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*, 2023.

David W. Romero, Anna Kuzina, Erik J. Bekkers, Jakub M. Tomczak, and Mark Hoogendoorn. CKConv: Continuous kernel convolution for sequential data. In *ICLR*, 2022.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 2014.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Roziere, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

Taha Vijay, Kashif Abid, Abdul Fatir Ansari, et al. GIFT-Eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*, 2024.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1945.

Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *ICML*, 2024.

Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *NeurIPS*, 2021.

Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. Timesnet: Temporal 2d-variation modeling for general time series analysis. In *ICLR*, 2023.

Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *AAAI*, 2023.

Yunhao Zhang and Junchi Yan. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *ICLR*, 2023.

Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI*, 2021.

Tian Zhou, Ziqing Ma, Qingsong Wen, Liang Sun, Tao Yao, Wotao Yin, and Rong Jin. FiLM: Frequency improved legendre memory model for long-term time series forecasting. In *NeurIPS*, 2022a.

Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *ICML*, 2022b.

Tian Zhou, PeiSong Niu, Xue Wang, Liang Sun, and Rong Jin. One fits all: Power general time series analysis by pretrained LM. In *NeurIPS*, 2023.

A Detailed Experimental Methodology

A.1 Training Protocol

All experiments use identical training infrastructure on NVIDIA H100/A100 GPUs via SLURM on the Princeton PLI/Della cluster. The complete training configuration:

Parameter	Value
Architecture	Moirai-2 Small (6 layers, $d = 384$, $d_{\text{ff}} = 1024$, $P = 16$)
Total parameters	11.4M (baseline), 11.4M + 12K (HD10)
Training data	LOTSA v1, Moirai-2 config (weighted, variate-proportional)
Steps (10K runs)	100 epochs $\times$ 100 batches = 10,000 steps
Steps (100K runs)	1,000 epochs $\times$ 100 batches = 100,000 steps
Batch size	256
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-6}$, wd $= 0.1$)
LR schedule	Cosine annealing, peak $\eta = 10^{-3}$, min $\eta = 0$
Warmup	1,000 steps (linear)
Precision	bf16 mixed precision
Anomaly filtering	z-score threshold = 8.0, variance ratio = 0.0 (disabled)
Random seeds	0, 1, 2, 7, 42 (5 seeds for main results)

A.2 Evaluation Protocol

All evaluations use the GIFT-Eval benchmark [Vijay et al., 2024] with 97 dataset$\times$horizon configurations:

- Context length: 4,000 tokens (maximum supported)
- Patch size: 32 for most frequencies, 8 for quarterly (following Moirai conventions)
- Batch size: 64
- Metric: MASE (Mean Absolute Scaled Error) per configuration
- Aggregation: geometric mean across 97 configurations
- Statistical test: paired sign test (binomial test, two-sided) on per-configuration wins
- Pooling: across seeds, giving $5 \times 97 = 485$ comparisons

A.3 Hint Channel Implementation

The Chebyshev polynomial $T_d(z)$ is expanded in the monomial basis to obtain FIR filter coefficients $c_0, \ldots, c_d$. The hint signal for time step t is:

$$\tilde{h}[t] = \left(\sum_{k=0}^{d} c_k \, x[t - ks] \right) - x[t]$$

This is computed over the full (normalized, zero-mean unit-variance) time series before patching. Unobserved time steps are masked to zero. During training with hint dropout p, each patch's hint independently has probability p of being replaced with zeros. At inference, the full hint is always provided.

The hint channel is concatenated with the target and mask channels before the input projection:

$$\mathbf{e}_i = \text{ResBlock}_{(2+n_h)P \to d}([\mathbf{p}_i; \mathbf{m}_i; \tilde{\mathbf{h}}_i^{(1)}; \ldots; \tilde{\mathbf{h}}_i^{(n_h)}])$$

where $n_h = 1$ for single-scale (HD10) and $n_h = 2$ for multi-scale (MSHD10). The ResBlock consists of:

$$\mathbf{z} = \text{SiLU}(W_1 \mathbf{x} + b_1) \qquad W_1 \in \mathbb{R}^{d \times (2+n_h)P} \tag{9}$$

$$\mathbf{o} = W_2 \mathbf{z} + b_2 + W_3 \mathbf{x} + b_3 \qquad W_2 \in \mathbb{R}^{d \times d},\ W_3 \in \mathbb{R}^{d \times (2+n_h)P} \tag{10}$$

Only W_1 and W_3 are wider than the baseline. W_2 and all downstream layers are identical.

B Per-Seed Detailed Results

Table 7: HD10 vs. Baseline: per-seed pairwise comparison (10K steps, $\eta = 10^{-3}$).

Seed	BL MASE	HD10 MASE	Δ%	Wins/97	p-value
0	0.8666	0.8234	+4.99%	66	2.5×10^{-4}
1	0.8503	0.8465	+0.44%	54	0.155
2	0.8630	0.8391	+2.77%	70	7.4×10^{-6}
7	0.8676	0.8320	+4.11%	78	5.8×10^{-10}
42	0.8614	0.8430	+2.13%	61	7.2×10^{-3}
Pooled			+2.89%	**329/485**	$\mathbf{1.5 \times 10^{-15}}$

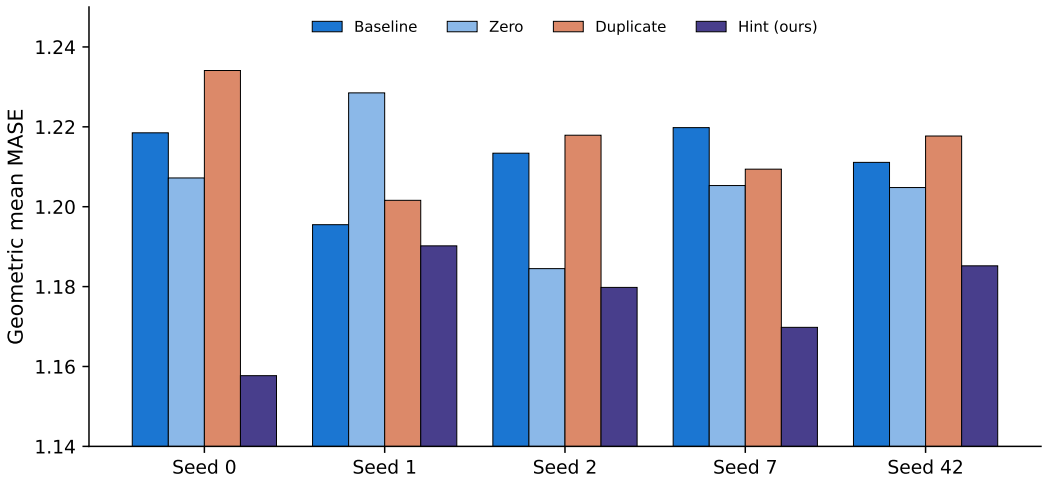

Figure 9: Per-seed MASE for all four conditions. HD10 (green) achieves the lowest MASE on every seed.

When averaging each configuration's MASE across all 5 seeds, HD10 wins 72/97 configurations ($p_{\text{sign}} = 9.6 \times 10^{-7}$; $p_{\text{Wilcoxon}} = 9.0 \times 10^{-9}$; $p_{t\text{-test}} = 2.4 \times 10^{-6}$). HD10 wins 3+ out of 5 seeds on 76.3% of configurations, and wins all 5 seeds on 23 configurations. Only 2 configurations (bizitobs_l2c/5T/medium, electricity/15T/short) see the baseline win all 5 seeds.

C Multi-Scale Variants

We test two multi-scale variants that provide two hint channels (degree 4 + degree 6):

Table 8: Multi-scale variants (5 seeds, $\eta = 10^{-3}$, 10K steps).

Method	**Mean**	**Std**	**vs BL**	**Wins/485**	p
HD10 (d=4, 10% drop)	0.8368	0.0082	−2.89%	329	1.5×10^{-15}
MS46 (d=4+6, 0% drop)	0.8434	0.0123	−2.13%	330	6.8×10^{-16}
MSHD10 (d=4+6, 10% drop)	0.8437	0.0072	−2.09%	315	2.2×10^{-11}

All three methods are statistically indistinguishable head-to-head (HD10 vs MS46: 251/485, $p = 0.47$; HD10 vs MSHD10: 257/485, $p = 0.20$). MS46 is the only method where all 5/5 individual seeds are significant. MSHD10 has the lowest cross-seed standard deviation (0.0072 vs HD10's 0.0082).

D Complete Experiment Catalog

We conducted 84 distinct training runs across the project. The following tables list all experiments with their configurations and results, organized by category. All results use geometric mean MASE on GIFT-Eval (97 configurations) at context length 4,000. "Wins" indicates configs where the method beats the matched-seed baseline. Boldface indicates the best in each category.

D.1 Capacity Controls (5 seeds, $\eta = 10^{-3}$, 10K steps)

Table 9: Capacity control experiments. Same architecture width, different channel content.

Method	**Content**	**Mean**	**Std**	**Wins/485**	p
HD10	Chebyshev $T_4(B^{16})$	**0.8368**	0.0082	**329**	1.5×10^{-15}
Zero	All zeros	0.8578	0.0102	262	0.042
Baseline	No extra channel	0.8617	0.0062	—	—
Duplicate	Copy of input	0.8650	0.0081	217	0.99

D.2 Polynomial Degree (seed 0, Chebyshev, $s = 16$, 10% dropout)

Degree	**Effective lags**	**MASE**	**Δ% vs BL**	**Wins/97**
2	1 (at $2P$)	0.8389	−3.20%	63
3	1 (at $3P$)	0.8498	−1.95%	55
4	2 (at $2P$, $4P$)	**0.8234**	−4.99%	**66**
5	2 (at P, $3P$, $5P$)	0.8444	−2.56%	—
6	3 (at $2P$, $4P$, $6P$)	0.8345	−3.71%	69
7	3 (at P, $3P$, $5P$, $7P$)	0.8397	−3.11%	—
8	4 (at $2P$–$8P$)	0.8413	−2.92%	62
0 (dup)	0	0.8583	−0.96%	42

D.3 Filter Stride (seed 0, Chebyshev $d = 4$, 10% dropout)

Stride	Lookback	MASE	Δ% vs BL	Wins/97
16 (= P)	32, 64 steps	**0.8234**	−4.99%	**66**
4	8, 16 steps	0.8407	−3.00%	57
8	16, 32 steps	0.8519	−1.71%	62
32	64, 128 steps	0.8485	−2.10%	63
8+16 (dual)	16, 32, 64 steps	0.8390	−3.20%	62

D.4 Filter Basis (seed 0, $d = 4$, $s = 16$, 10% dropout)

Basis	MASE	Δ%	Wins vs BL	p	vs Cheby	p
Chebyshev	**0.8234**	−4.99%	66/97	2.5×10^{-4}	—	—
EMA	0.8329	−3.89%	65/97	5.3×10^{-4}	41/97	0.077
Legendre $d = 4$	0.8400	−3.07%	62/97	4.0×10^{-3}	30/97	<0.001
Legendre $d = 6$	0.8397	−3.10%	64/97	1.1×10^{-3}	50/97	0.42
Finite Diff	0.8569	−1.12%	54/97	0.16	29/97	<0.001
Duplicate	0.8583	−0.96%	42/97	0.11	27/97	<0.001

D.5 Hint Dropout Rate (seed 0, Chebyshev $d = 4$, $s = 16$)

Dropout	MASE	Δ% vs BL	Wins/97	p
0% (HD0)†	0.8378	−3.5%	—	—
2%	0.8389	−3.20%	63	0.004
3%	0.8498	−1.95%	55	0.22
10%	**0.8234**	**−4.99%**	**66**	**0.0005**
20%	0.8423	−2.81%	67	0.0002
30%	0.8559	−1.23%	61	0.014

†HD0 MASE is the 5-seed mean (per-seed sign test not available). HD0 and HD10 are essentially tied (0.8378 vs. 0.8368); dropout helps marginally.

D.6 Inference-Time Content Ablation (seed 0, HD10 model)

A model trained with Chebyshev hints is evaluated with different hint content at inference:

D.7 Learned FIR Coefficients (seed 0)

Instead of fixed Chebyshev coefficients, the filter weights are made learnable:

The two learned filters converge to distinct basins (high-pass vs. smoothing). Both beat the baseline significantly but are slightly worse than the fixed Chebyshev (51/97 h2h, $p = 0.34$).

D.8 Latent Preconditioning (seed 0)

Applying the polynomial filter *after* the input projection (in latent space) rather than before patching:

Inference hint content	MASE	vs Normal
Normal (Chebyshev)	0.8234	—
No hint channel (Baseline arch)	0.8666	+5.2%
Zeros	0.9003	+9.3%
Random Gaussian noise	0.9427	+14.5%
Duplicate (copy of input)	1.0699	+29.9%

Initialization	Learned coefficients	MASE	Δ% vs BL
Chebyshev init	$[-0.026, -1.032, -0.123, 0.007]$	0.8332	−3.85%
Zero init	$[0.308, 0.177, 0.057, 0.045]$	0.8358	−3.57%
Fixed Chebyshev	$[0, -2, 0, 0.5]$ (not learned)	0.8234	−4.99%

Latent preconditioning is catastrophic. The filter must operate on the raw time series, not on learned embeddings.

D.9 Context Length (seed 0)

D.10 100K Training (5 seeds, with capacity controls)

Baseline seed 0 exhibits training instability at 80K steps (MASE spikes to 0.9273 before partial recovery to 0.9092); seeds 2 and 7 are stable (∼0.86). Duplicate *hurts* at 100K (−0.9% vs BL), and Zero hurts more (−1.7%; note seed 42 outlier at 0.9610). HD10 remains the most stable method across seeds (range 0.83–0.86 vs. BL range 0.86–0.91).

D.11 Training dynamics

The training loss curves reveal several insights about how the hint channel affects optimization dynamics.

Slow start, growing gap. Figure 10 and Table 11 show the 10K-step comparison. HD10 starts with *higher* training loss than the baseline for the first ∼200–300 epochs (−0.6% at epoch 100), then crosses over and gradually builds an advantage that reaches +1.0% by epoch 10,000. This "slow start" is consistent across all 5 seeds (crossover at epochs 99–299) and suggests the model must first learn basic patch representations before it can exploit the polynomial structure in the hint channel.

Monotonically growing gap at 100K. At the full training budget (Table 13, Figure 11), the HD10 training loss advantage grows monotonically throughout training and *never plateaus*: −1.1% at epoch 100 (HD10 behind), +0.9% at 5K, +1.9% at 30K, +2.7% at 70K, and +3.5% at 100K. This suggests the hint channel provides compounding benefits with longer training, and the method would likely improve further beyond 100K steps.

Optimization smoothing. HD10 exhibits lower epoch-to-epoch training loss volatility than the baseline (0.0124 vs. 0.0138, averaged across 5 seeds), with much more consistent variance across seeds (std 0.0002 vs. 0.003). The polynomial hint channel acts as an implicit regularizer that smooths the optimization landscape, reducing sensitivity to random initialization.

Method	Filter location	MASE	vs BL
HD10 (input-level)	Before patching	0.8234	−4.99%
Latent $s = 1$	After input projection	1.1539	+33.1%
Latent $s = 4$	After input projection	1.2244	+41.3%

Context	BL	HD10	Δ%	Wins/97
512	0.9432	0.8986	−4.73%	62
1,000	0.9166	0.8706	−5.02%	65
2,000	0.8813	0.8331	−5.47%	71
4,000	0.8666	0.8234	−4.99%	66

Late-training generalization divergence. At 100K steps, all methods—including HD10—show a small training loss increase in the final epochs (0.3–1.3% above the per-run minimum, which occurs around epoch 92K–98K). Critically, this mild training loss uptick affects all methods similarly, yet the *eval MASE* diverges sharply: the baseline's eval degrades much more than HD10's. This confirms that the hint channel's primary benefit is *generalization*, not optimization: the polynomial structure provides an inductive bias that maintains eval performance even as training loss slightly deteriorates in the cosine schedule's final phase.

Table 14 quantifies this train–eval amplification. At 10K, the training loss gap (−1.0%) is amplified 2.9× into a −2.9% eval improvement. At 100K, the gap grows to −2.4% training and −3.9% eval (1.6× amplification). The capacity controls show the reverse: zero and duplicate channels have training loss comparable to baseline but *worse* eval MASE at 100K—uninformative channels hurt generalization despite not affecting optimization.

D.12 Matched Learning Rate (5 seeds, $\eta = 2 \times 10^{-3}$)

Table 16 shows the training loss at $\eta = 2 \times 10^{-3}$. At this higher learning rate, HD10 still achieves lower training loss (mean 0.1339 vs 0.1353, −1.0%), with a matching eval MASE improvement of +0.72%. The training loss advantage of the hint channel is remarkably consistent across learning rates.

D.13 Base Model Scale (46M parameters, 10K steps)

Results are inconsistent at the 46M scale. The hint channel's benefit appears specific to capacity-constrained models.

D.14 Prediction Horizon Effect (seed 0)

D.15 Model Soup (seed 0)

Weight averaging between baseline and HD10 checkpoints:

D.16 Zero-Hint Dependence vs Dropout Rate (seed 0)

Models trained with different hint dropout rates, evaluated with hint channel replaced by zeros:

Without dropout, the model is catastrophically dependent on hints (36% degradation when removed). With 10% dropout, degradation is moderate (9%), and with 30% the model barely uses hints at all (2.5% degradation but weaker overall performance).

Table 10: Full 100K training budget with capacity controls (5 seeds). HD10 vs BL: +3.9%; HD10 vs Dup: +4.7%; HD10 vs Zero: +5.5%.

	s0	**s1**	**s2**	**s7**	**s42**	**Mean**
Baseline	0.9092	0.8920	0.8633	0.8641	0.8636	0.8784
HD10	0.8349	0.8402	0.8586	0.8450	0.8430	0.8443
MSHD10	0.8266	—	0.8432	0.8437	—	0.8378
Duplicate	0.8887	0.8740	0.8974	0.8794	0.8903	0.8860
Zero	0.8917	0.8585	0.8750	0.8825	0.9610	0.8937
HD10 Δ% vs BL	−8.2%	−5.8%	−0.5%	−2.2%	−2.4%	−3.9%

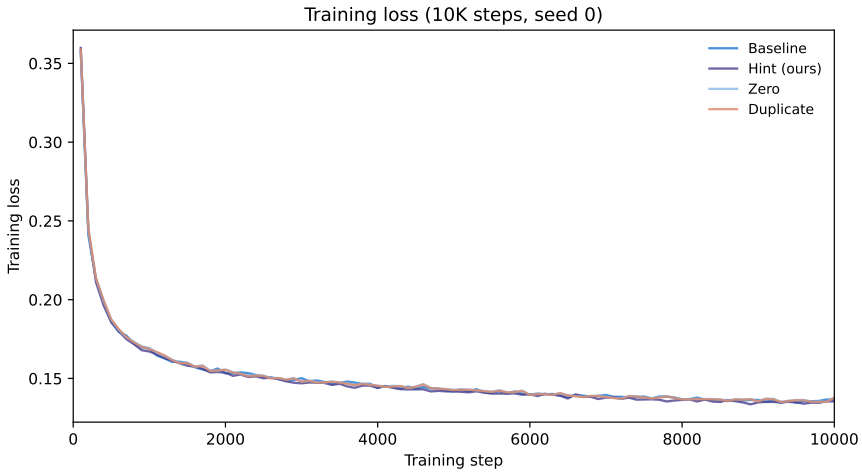

Figure 10: Training loss at 10K steps (seed 0). HD10 starts slightly worse than baseline, crosses over around epoch 300, and builds a growing advantage.

D.17 Computational Overhead

Measured across 9 paired runs (HD10 and BL on the same GPU node):

D.18 Seed Consistency Distribution

D.19 Representative Configurations

D.20 Full Configuration Heatmap

D.21 FEV-Bench (Second Benchmark)

To verify that the improvement is not specific to GIFT-Eval, we evaluate on the FEV-Bench [Vijay et al., 2024] forecasting benchmark, which comprises 100 real-world forecasting tasks from domains including energy pricing, e-commerce, weather, and power systems.

HD10 improves over the baseline by −2.5% MASE and −1.3% SQL (scaled quantile loss), winning 72 of 100 tasks on both metrics. This is consistent with the −2.9% improvement on GIFT-Eval, confirming the benefit transfers across benchmarks.

Table 11: Final training loss across 5 seeds at 10K steps ($\eta = 10^{-3}$). Training loss differences are small (∼1%), while eval MASE differences are much larger (∼3%), confirming hints act as a regularizer.

Method	**s0**	**s1**	**s2**	**s7**	**s42**	**Mean**	**Std**	**Eval MASE**
Baseline	.1366	.1365	.1358	.1366	.1365	.1364	.0003	0.8617
HD10 (ours)	.1354	.1350	.1350	.1353	.1343	**.1350**	.0004	**0.8368**
Zero	.1378	.1369	.1363	.1366	.1353	.1366	.0008	0.8578
Duplicate	.1376	.1366	.1370	.1365	.1355	.1366	.0007	0.8650
HD10 Δ% vs BL						−1.0%		−2.9%

Table 12: HD10 vs baseline training loss gap over 100K steps (seed 0). The gap grows monotonically and does not plateau.

Epoch	100	5K	10K	30K	50K	70K	100K
BL loss	.3600	.1441	.1405	.1318	.1267	.1238	.1185
HD10 loss	.3641	.1428	.1387	.1293	.1240	.1205	.1144
Gap	−1.1%	+0.9%	+1.3%	+1.9%	+2.2%	+2.7%	+3.5%

D.22 Leaderboard Context

Table 18 places our models in the context of the GIFT-Eval leaderboard. Our retrained HD10 (11.4M parameters, 10K training steps) outperforms the official Moirai 1.0 Large (311M parameters) despite having 27× fewer parameters and a fraction of the training compute.

D.23 Matched-Official Protocol (100K steps, 10K warmup)

The 100K results in the main paper use a 1,000-step warmup, while the official Moirai 2.0 training protocol [Liu et al., 2024a] uses 10,000 steps of warmup. To control for this protocol difference, we retrain both baseline and HD10 at 100K steps with the exact official hyperparameters.

With 10K warmup (matching the official protocol), HD10 still outperforms the baseline by 2.3% on average across 3 seeds (compared to 3.9% with 1K warmup). The smaller advantage reflects that the longer warmup partially stabilizes the baseline—at seed 0, BL improves from 0.909 (1K warmup, which shows the late-training collapse) to 0.882 (10K warmup). The benefit is seed-dependent: HD10 wins at seed 2 (+3.8%) and seed 7 (+4.0%), but slightly loses at seed 0 (−1.1%). This suggests that part of HD10's 100K advantage at 1K warmup is robustness to schedule-induced instability, but a genuine ∼2% inductive-bias effect persists under the matched-official protocol.

Table 13: Final training loss across seeds at 100K steps ($\eta = 10^{-3}$). HD10 maintains lower training loss on every seed. Zero and Duplicate have training loss comparable to baseline despite worse eval MASE.

Method	s0	s1	s2	s7	s42	Mean	Std	Eval MASE
Baseline	.1185	.1185	.1190	.1194	.1179	.1187	.0005	0.8784
HD10 (ours)	.1144	.1170	.1155	.1172	.1155	**.1159**	.0010	**0.8443**
Zero	.1186	.1188	.1184	.1188	.1187	.1187	.0002	0.8937
Duplicate	.1192	.1193	.1185	.1197	.1182	.1190	.0006	0.8860
MSHD10	.1147	—	.1145	.1145	—	.1146	.0001	0.8378[†]
HD10 Δ% vs BL						−2.4%		−3.9%

[†]MSHD10 eval mean over 3 available seeds (0, 2, 7).

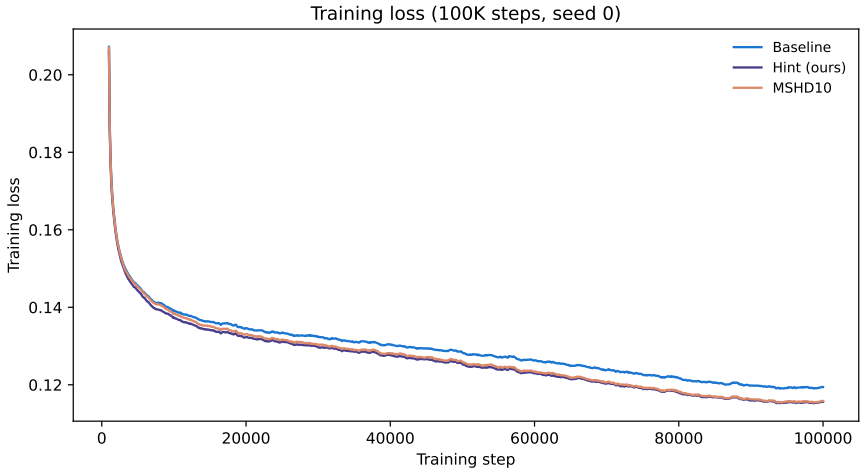

Figure 11: Training loss at 100K steps (seed 0). The HD10 advantage grows monotonically throughout training (+0.9% at 5K to +3.5% at 100K).

Table 14: Train loss vs eval MASE improvement (5-seed mean, HD10 over baseline).

	Train loss Δ%	Eval MASE Δ%	Amplification
10K steps (5 seeds)	−1.0%	−2.9%	2.9×
100K steps (5 seeds)	−2.4%	−3.9%	1.6×

Table 15: BL and HD10 at matched learning rate $\eta = 2 \times 10^{-3}$.

Seed	BL@2e-3	HD10@2e-3	Δ%	Wins/97	p-value
0	0.8424	0.8339	+1.01%	57	0.052
1	0.8442	0.8338	+1.23%	55	0.111
2	0.8378	0.8468	−1.07%	46	0.729
7	0.8339	0.8350	−0.13%	54	0.155
42	0.8526	0.8307	+2.55%	68	4.7×10^{-5}
Pooled	**0.8422**	**0.8361**	+0.72%	**280/485**	3.8×10^{-4}

Table 16: Training loss at matched LR $\eta = 2 \times 10^{-3}$ (5 seeds, 10K steps).

Method	**s0**	**s1**	**s2**	**s7**	**s42**	**Mean**
BL@2e-3	.1355	.1359	.1357	.1349	.1345	.1353
HD10@2e-3	.1346	.1341	.1340	.1337	.1331	.1339
Δ%	-0.7%	-1.3%	-1.2%	-0.9%	-1.1%	-1.0%

Seed	**BL (46M)**	**HD10 (46M)**	**Δ%**	**Wins**
0	0.8504	0.8520	-0.19%	52/95
1	0.8473	0.8509	-0.42%	60/95
42	0.8570	0.8417	$+1.80$%	59/93

Prediction length	**HD10 Δ%**	**Win rate**
30	-0.1%	57%
48	-2.1%	70%
480	-6.7%	68%
720	-8.6%	79%
900	-14.1%	100%

Method	**MASE**
Baseline	0.8666
HD10	0.8234
Soup ($\alpha = 0.5$)	0.8307
Soup ($\alpha = 0.7$)	0.8250

Train dropout	**Normal MASE**	**Zero-hint MASE**	**Degradation**
0% (MS46)	0.8533	1.1594	+35.9%
2%	0.8389	0.8996	+7.2%
10% (HD10)	0.8234	0.9003	+9.3%
30%	0.8559	0.8769	+2.5%

	Baseline	**HD10**
Training throughput	2.18 it/s	2.14 it/s (-1.6%)
Extra parameters	0	12,288 (+0.11%)
Peak GPU memory	identical	identical

Table 17: FEV-Bench results (100 tasks, seed 0, 10K steps). HD10 wins 72% of tasks on this independent benchmark.

Method	**MASE (geo)**	**SQL (mean)**	**MASE wins**	**SQL wins**
Baseline	1.2841	1.6786	—	—
HD10 (Ours)	**1.2525**	**1.6572**	**72/100 (72%)**	**72/100 (72%)**
Δ%	-2.5%	-1.3%		

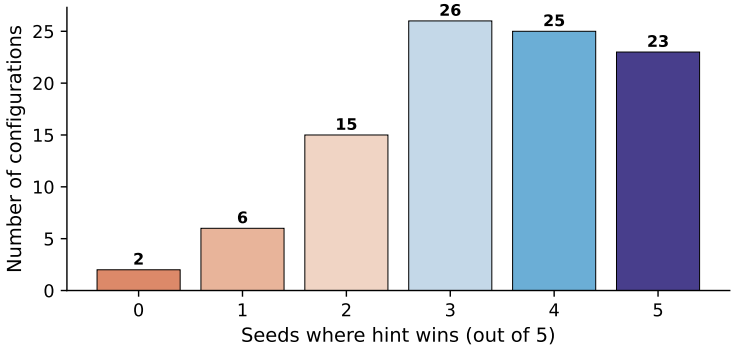

Figure 12: For each configuration, how many of 5 seeds does HD10 win? 74 of 97 configs see HD10 win the majority ($\geq 3/5$) of seeds.

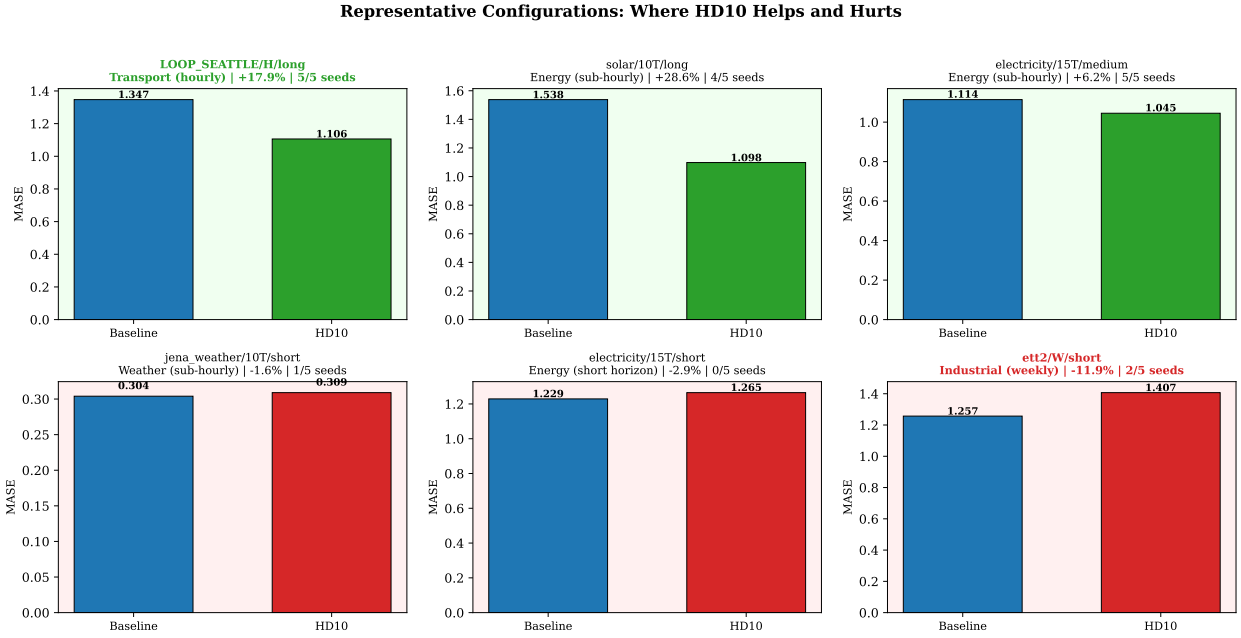

Figure 13: Six representative configurations spanning the range of HD10 performance. Top row: strong wins (transport, energy). Bottom row: neutral (weather) and losses (short horizon, weekly).

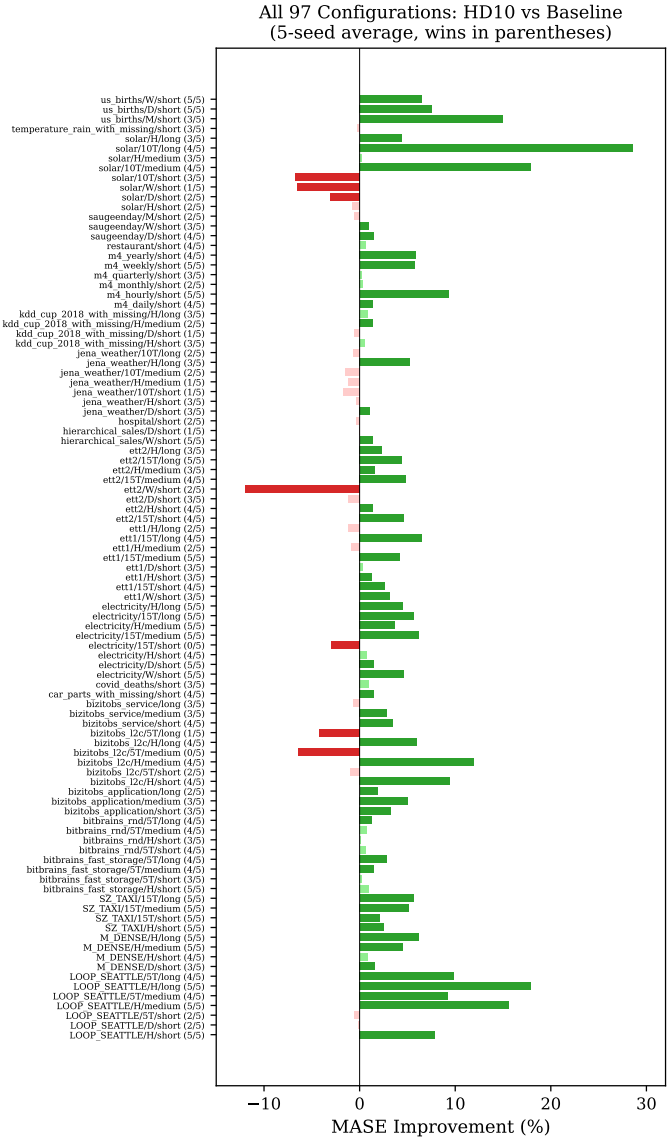

Figure 14: All 97 configurations sorted by dataset, showing 5-seed averaged improvement and seed win count. Dark green: strong improvement. Red: regression.

Table 18: GIFT-Eval leaderboard context. Our HD10 at 11.4M params outperforms Moirai 1.0 Large at 311M.

Model	**Params**	**Norm. MASE**
Moirai 2.0-R-Small (official)	11.4M	0.728
Kairos-50M	53M	0.742
TimesFM-2.0	500M	0.758
ChronosBolt-Base	205M	0.808
Our HD10 (seed 0, 10K)	**11.4M**	**0.828**
PatchTST (leaderboard)	—	0.849
Our Baseline (seed 0, 10K)	11.4M	0.870
Moirai 1.0 Large	311M	0.875
Moirai 1.0 Base	90M	0.901
Moirai 1.0 Small	14M	0.946

Table 19: Matched-official 100K results (10K warmup, cosine annealing to 0, otherwise identical to official protocol).

Method	**s0**	**s2**	**s7**	**Mean**	**vs BL**
Baseline (1K warmup)	0.909	0.863	0.864	0.878	—
Baseline (10K warmup)	0.882	0.880	0.917	0.893	—
HD10 (1K warmup)	0.835	0.859	0.845	0.844	−3.9%
HD10 (10K warmup)	0.892	0.846	0.880	**0.873**	**−2.3**%